

tic_tac_toe 보고서

인공지능학부
234890 김연우

1. 서론

1. 프로젝트 목적 및 배경

본 프로젝트는 C++ 프로그래밍 수업에서 4주차까지 학습한 배열, 반복문, 조건문, switch문, 입출력 등의 프로그래밍 요소를 활용하여 간단한 tic tac toe 게임을 구현하였다.

2. 목표

두 명의 사용자가 번갈아 X와 O를 게임판에 놓고 가로, 세로, 대각선 중 한 줄을 먼저 완성하면 승리하는 tic tac toe 게임을 구현하는 것을 목표로 한다.

2. 요구사항

1. 사용자 요구사항

- 두 명의 사용자가 번갈아 가며 게임 진행
- 사용자는 (x, y) 좌표를 입력

2. 기능 요구사항

- 누구의 차례인지 출력
- 좌표 입력 받기
- 입력 받은 좌표 유효성 체크
- 좌표에 O/X 놓기
- 현재 보드판 출력
- 빙고 시 승자 출력 후 종료
- 모든 칸이 찼으면 종료
- 제약 조건 : 보드판은 2차원 배열 사용

3. 설계 및 구현

1. 기능 별 구현 사항

1) 누구의 차례인지 출력

```
16 // 게임 코드
17 int k = 0;
18 char currentUser = 'X';
19 while(true) {
20     // 1. 누구 차례인지 출력
21     switch (k % 2) {
22     case 0:
23         cout << (k % 2) + 1 << "번 유저(X)의 차례입니다 -> ";
24         currentUser = 'X';
25         break;
26     case 1:
27         cout << (k % 2) + 1 << "번 유저(O)의 차례입니다 -> ";
28         currentUser = 'O';
29         break;
30     }
```

입력

- K%2 : 현재 사용자를 결정하기 위한 값

결과

- 1번 유저(X)의 차례인지, 2번 유저(O)의 차례인지 출력

설명

1. K%2의 값을 switch로 확인
2. K%2의 값이 0이면 1번 유저(X)의 차례를 출력
3. K%2의 값이 1이면 2번 유저(O)의 차례를 출력

2) 좌표 입력 받기

```
32 // 2. 좌표 입력 받기
33 cout << "(x, y) 좌표를 입력하세요: ";
34 cin >> x >> y;
```

입력

- X : 좌표 x 값
- Y : 좌표 y 값

결과

- 사용자가 입력한 x와 y 좌표를 저장

설명

1. cin을 통해 x와 y 값을 입력받음

3) 입력 받은 좌표 유효성 체크

```
36 // 3. 입력받은 좌표의 유효성 체크
37 if (x >= numCell || y >= numCell) {
38     cout << x << ", " << y << ": ";
39     cout << "x와 y 둘 중 하나가 칸을 벗어납니다." << endl;
40     continue;
41 }
42 if (board[x][y] != ' ') {
43     cout << x << ", " << y << ": 이미 돌아 차있습니다." << endl;
44     continue;
45 }
```

입력

- X : 좌표 x 값
- Y : 좌표 y 값
- numCell : 가로/세로 칸 개수
- board 배열 : 현재 게임판

결과

- 칸을 놓을 수 없는 이유를 출력

설명

1. 사용자가 입력한 좌표가 게임판을 벗어나는지 if문 확인
2. 사용자가 입력한 좌표에 돌이 이미 있는지 if문 확인
3. 조건에 해당하면 continue를 통해 while문 초반으로 이동

4) 좌표에 O/X 놓기

```
47 // 4. 입력받은 좌표에 현재 유저의 돌 놓기
48 board[x][y] = currentUser;
```

입력

- X : 좌표 x 값
- Y : 좌표 y 값
- currentUser : 현재 사용자

결과

- 입력받은 좌표에 현재 사용자의 돌(O/X)을 놓음

설명

1. 입력받은 좌표 x, y 값을 통해 board[x][y]에 현재 사용자의 돌을 저장

5) 현재 보드판 출력

```
50 // 5. 현재 보드판 출력
51 for (int i = 0; i < numCell; i++) {
52     cout << "---|---|---" << endl;
53     for (int j = 0; j < numCell; j++) {
54         cout << board[i][j];
55         if (j == numCell - 1) {
56             break;
57         }
58         cout << " |";
59     }
60     cout << endl;
61 }
62 cout << "---|---|---" << endl;
```

입력

- board 배열 : 현재 게임판
- numCell : 가로/세로 칸 개수

결과

- 현재 게임판의 상태 출력

설명

1. For문을 통해 게임판의 행을 확인
2. 중첩 for문을 통해 행의 각 칸의 내용(board 배열)을 출력
3. | 와 ---을 통해 게임판 형태로 출력

6) 모든 칸이 찼으면 종료

```
64 // 6. 모든 칸이 차면 종료
65 int checked = 0;
66 for (int i = 0; i < numCell; i++) {
67     for (int j = 0; j < numCell; j++) {
68         if (board[i][j] != ' ') {
69             checked++;
70         }
71     }
72 }
73 if (checked == numCell * numCell) {
74     cout << "모든 칸이 다 찼습니다. 종료합니다." << endl;
75     break;
76 }
```

입력

- board 배열 : 현재 게임판
- numCell : 가로/세로 칸 개수
- checked : 돌이 놓인 칸의 개수

결과

- 모든 칸이 차면 종료 메시지 출력
- while문 종료

설명

1. checked 초기화
2. 중첩 for문 통해 모든 게임판의 모든 칸(board 배열) 확인
3. board 배열에 공백이 아니면(돌이 놓여있으면) checked 증가
4. checked 값이 전체 칸의 개수(numCell*numCell)와 동일하면 break 통해 while문 빠져나와 게임 종료

7) 현재 보드판 출력

```
78 // 7. 빙고 시 승리 출력 후 종료 (가로, 세로, 대각선)
79 bool bingo = false;
80
81 for (int i = 0; i < numCell; i++) {
82     // 가로줄 빙고 확인
83     if (board[i][0] == currentUser && board[i][1] == currentUser && board[i][2] == currentUser) {
84         cout << "가로에 모두 돌이 놓였습니다! ";
85         bingo = true;
86     }
87     // 세로줄 빙고 확인
88     if (board[0][i] == currentUser && board[1][i] == currentUser && board[2][i] == currentUser) {
89         cout << "세로에 모두 돌이 놓였습니다! ";
90         bingo = true;
91     }
92 }
93 // 대각선 빙고 확인
94 if (board[0][0] == currentUser && board[1][1] == currentUser && board[2][2] == currentUser) {
95     cout << "왼쪽 위에서 오른쪽 아래 대각선으로 모두 돌이 놓였습니다! ";
96     bingo = true;
97 }
98 if (board[0][2] == currentUser && board[1][1] == currentUser && board[2][0] == currentUser) {
99     cout << "오른쪽 위에서 왼쪽 아래 대각선으로 모두 돌이 놓였습니다! ";
100     bingo = true;
101 }
102
103 if (bingo) {
104     cout << (k % 2) + 1 << "번 유저(" << currentUser << ")의 승리입니다!" << endl;
105     cout << "종료합니다." << endl;
106     break;
107 }
108 k++;
```

입력

- board 배열 : 현재 게임판
- currentUser : 현재 사용자
- bingo : 빙고의 유무

결과

- 가로/세로/대각선에 빙고가 되었는지 출력
- 빙고가 되면 빙고 메시지 출력 후 승리한 사용자와 종료 메시지 출력

설명

1. for문을 통해 가로(행)와 세로(열)를 확인
2. 같은 사용자의 돌이 한 줄에 모두 있는지 if문을 통해 확인
3. if문을 true로 통과하면 빙고 메시지 출력 후 bingo를 true로 변경
4. 왼쪽 위에서 오른쪽 아래 방향의 대각선/오른쪽 위에서 왼쪽 아래 방향의 대각선을 if문을 통해 확인
5. 각각 if문을 true로 통과하면 빙고 메시지 출력 후 bingo를 true로 변경
6. Bingo가 true인지 if문을 통해 확인
7. Bingo가 true이면 현재 사용자의 승리 메시지와 종료 메시지 출력
8. Break문을 통해 while문을 빠져나와 게임 종료

9. Bingo가 false라면 k를 1 증가시켜 다음 사용자로 전환

4. 테스트

1. 기능 별 테스트 결과

- 누구의 차례인지 출력

```
1번 유저(X)의 차례입니다 -> (x, y) 좌표를 입력하세요:
```

```
2번 유저(O)의 차례입니다 -> (x, y) 좌표를 입력하세요: █
```

- 좌표 입력 받기

```
1번 유저(X)의 차례입니다 -> (x, y) 좌표를 입력하세요: 0 2
```

```
2번 유저(O)의 차례입니다 -> (x, y) 좌표를 입력하세요: 0 1
```

- 입력 받은 좌표 유효성 체크

```
1번 유저(X)의 차례입니다 -> (x, y) 좌표를 입력하세요: 0 4
0, 4: x와 y 둘 중 하나가 칸을 벗어납니다.
```

```
1번 유저(X)의 차례입니다 -> (x, y) 좌표를 입력하세요: 0 1
0, 1: 이미 둘이 차있습니다.
```

- 좌표에 O/X 놓기, 현재 보드판 출력

```
1번 유저(X)의 차례입니다 -> (x, y) 좌표를 입력하세요: 0 2
---|---|---
  |  |X
---|---|---
  |  |
---|---|---
  |  |
---|---|---
```

```
2번 유저(O)의 차례입니다 -> (x, y) 좌표를 입력하세요: 0 1
---|---|---
  |O |X
---|---|---
  |  |
---|---|---
  |  |
---|---|---
```

- 모든 칸이 찼으면 종료

```
1번 유저(X)의 차례입니다 -> (x, y) 좌표를 입력하세요: 1 2
---|---|---
X |X |O
---|---|---
O |O |X
---|---|---
X |O |X
---|---|---
모든 칸이 다 찼습니다. 종료합니다.
```

- 빙고 시 승자 출력 후 종료

```
1번 유저(X)의 차례입니다 -> (x, y) 좌표를 입력하세요: 2 0
---|---|---
X  |O  |X
---|---|---
    |X  |O
---|---|---
X  |   |O
---|---|---
오른쪽 위에서 왼쪽 아래 대각선으로 모두 돌이 놓였습니다!: 1번 유저(X)의 승리입니다!
종료합니다.
```

2. 최종 테스트 스크린샷

```
1번 유저(X)의 차례입니다 -> (x, y) 좌표를 입력하세요: 2 0
---|---|---
X  |O  |X
---|---|---
    |X  |O
---|---|---
X  |   |O
---|---|---
오른쪽 위에서 왼쪽 아래 대각선으로 모두 돌이 놓였습니다!: 1번 유저(X)의 승리입니다!
종료합니다.
```

5. 결과 및 결론

1. 프로젝트 결과

C++의 배열, 반복문, 조건문, switch문, while문, 입출력 등을 활용하여 tic tac toe 게임을 구현하였다.

2. 느낀점

이번 프로젝트를 통해 수업에서 배운 문법을 실제 프로그램에 적용해 볼 수 있어 흥미로웠고, 프로그램 구현 과정과 테스트 과정을 보고서를 통해 체계적으로 정리함으로써 각 기능이 어떻게 동작하고 연결되는지 더 명확히 이해할 수 있었다.